Project – Low level Design on IaC Provisioning for Non-Profit System

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	Neha Tamboli	EN22CS301641
2	Riddhima Taose	EN22CS303042
3	Uday Patidar	EN23CS3L1023
4	Rohan Rawat	EN23CS3L1016
5	Naman Dwivedi	EN22CS301627

Group Name: 04D6

Project Number: DO-15

Industry Mentor Name:

University Mentor Name: Akshay Saxena

Academic Year: 2025-26

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. **References**

1. Introduction

1.1. Scope of the document

This Low Level Design (LLD) document provides comprehensive technical specifications and implementation details for the AI-Based Non-Profit Support Triage System. The document focuses on detailed architecture, component interactions, database schemas, API endpoints, containerization strategies, and Infrastructure as Code (IaC) implementation using Docker, Kubernetes, and CI/CD pipelines.

The scope encompasses:

- Detailed system architecture and component design for the AI-powered triage system
- Database schema design for message processing, user management, and analytics
- RESTful API specifications and endpoints for message submission and classification
- Docker containerization architecture using multi-stage builds
- Kubernetes orchestration configuration for scalable deployment
- LangChain agent workflow implementation for AI processing
- ChromaDB vector database integration for semantic search
- CI/CD pipeline design using Jenkins or GitHub Actions
- Infrastructure provisioning using Terraform (if applicable)
- Security mechanisms including authentication, data encryption, and API key management
- Monitoring and logging infrastructure for production environments

This document serves as the definitive technical reference for developers implementing the system and provides deployment guidelines for DevOps engineers managing the infrastructure.

1.2 Intended Audience

This document is intended for the following stakeholders:

- **Backend Developers**: For implementing Python-based AI agents, LangChain workflows, and API endpoints
- **AI/ML Engineers**: For understanding NLP model integration, prompt engineering, and LLM orchestration
- **DevOps Engineers**: For containerization, Kubernetes deployment, and CI/CD pipeline configuration
- **Database Administrators**: For database schema implementation, vector database optimization, and data migration
- **Frontend Developers**: For Streamlit interface development and API integration

- **QA Engineers**: For testing API endpoints, AI agent behaviors, and containerized deployments
- **Security Engineers**: For implementing authentication mechanisms, securing API keys, and vulnerability assessments
- **Project Mentors and Supervisors**: For technical review, architecture validation, and assessment
- **System Architects**: For understanding design patterns, scalability strategies, and technology choices

1.3 System Overview

The AI-Based Non-Profit Support Triage System is an intelligent automation platform designed to streamline communication processing for non-profit organizations. The system receives messages from multiple stakeholder groups including donors, volunteers, and beneficiaries, and automatically performs intelligent triage operations using advanced Natural Language Processing and Large Language Models.

Core Functionalities:

1. **Message Classification**: Automatically categorizes incoming messages into predefined classes such as donation inquiries, volunteer requests, emergency assistance, general information, and complaints
2. **Urgency Detection**: Analyzes message content to determine priority levels (Critical, High, Medium, Low) based on keywords, sentiment, and contextual understanding
3. **Entity Extraction**: Identifies and extracts key information including donor names, donation amounts, contact information, dates, and location references
4. **Response Generation**: Creates contextually appropriate draft responses using LLM-powered generation, reducing manual response time
5. **Semantic Search**: Enables retrieval of similar past messages and responses using vector embeddings stored in ChromaDB
6. **Analytics Dashboard**: Provides insights on message volume, classification trends, response times, and operational metrics4

Technology Architecture:

The system leverages a modern DevOps-centric architecture built on containerization and Infrastructure as Code principles:

- **Frontend Layer**: Streamlit-based interactive web interface providing intuitive message submission forms, classification results display, and administrative dashboards
- **Backend Layer**: Python-based application server using LangChain framework for AI agent orchestration, FastAPI or Flask for RESTful API endpoints

- **AI Processing Layer**: Integration with Large Language Models (OpenAI GPT, Anthropic Claude, or local models) through LangChain agents for classification, extraction, and generation tasks
- **Vector Database**: ChromaDB for storing message embeddings and enabling semantic similarity search
- **Containerization**: Docker containers with multi-stage builds for optimized image size and security isolation
- **Orchestration**: Docker Compose for local development and Kubernetes for production-grade deployment with auto-scaling and load balancing
- **CI/CD Pipeline**: Automated build, test, and deployment workflows using Jenkins or GitHub Actions

2. System Design

2.1 Application Design

The AI-Based Non-Profit Support Triage System follows a layered microservices architecture with clear separation of concerns and modular design principles.

Architecture Pattern:

The application implements a **three-tier architecture** with AI processing layer:

1. **Presentation Layer**: Streamlit web interface for user interaction
2. **API Layer**: RESTful API endpoints for client-server communication
3. **Business Logic Layer**: AI agent orchestration, message processing, and workflow management
4. **Data Access Layer**: Database operations, vector storage, and data persistence

2.2 Process Flow

End-to-End Message Processing Flow:

1. **Message Submission**:
    - User accesses Streamlit interface
    - Fills message submission form (sender info, message content, contact details)
    - Frontend validates input and sends POST request to /api/messages endpoint
    - API Gateway receives request and authenticates user (if required)
2. **Message Storage**:
    - Message service validates request payload

- Message record created in primary database (PostgreSQL/MongoDB)
- Unique message ID generated and returned to client
- Message status set to "pending_classification"

3. **AI Classification Process**:
    - Classification agent retrieves message content
    - Message text preprocessed (cleaning, normalization)
    - LangChain agent invokes LLM with classification prompt
    - LLM returns predicted class (Donation, Volunteer, Emergency, Info, Complaint)
    - Confidence score calculated and stored
4. **Urgency Detection**:
    - Urgency detection agent analyzes message for priority keywords
    - Sentiment analysis performed to detect emotional urgency
    - Urgency level assigned (Critical, High, Medium, Low)
    - Automatic escalation triggered for Critical/High urgency messages
5. **Entity Extraction**:
    - Extraction agent processes message content
    - LLM extracts structured entities (names, amounts, dates, locations)
    - Entities validated against predefined schemas
    - Extracted data stored in structured format
6. **Vector Embedding Generation**:
    - Message text converted to vector embedding using embedding model
    - Embedding stored in ChromaDB with metadata
    - Enables future semantic similarity search
7. **Response Generation**:
    - Generation agent retrieves message classification and entities
    - Similar past responses retrieved from ChromaDB
    - LLM generates contextually appropriate draft response
    - Draft saved for human review and editing
8. **Result Presentation**:
    - Processing results returned to frontend via API
    - Streamlit displays classification, urgency, entities, and draft response
    - Human operator can review, edit, and approve response
    - Final response sent to message sender

Docker Container Lifecycle Flow:

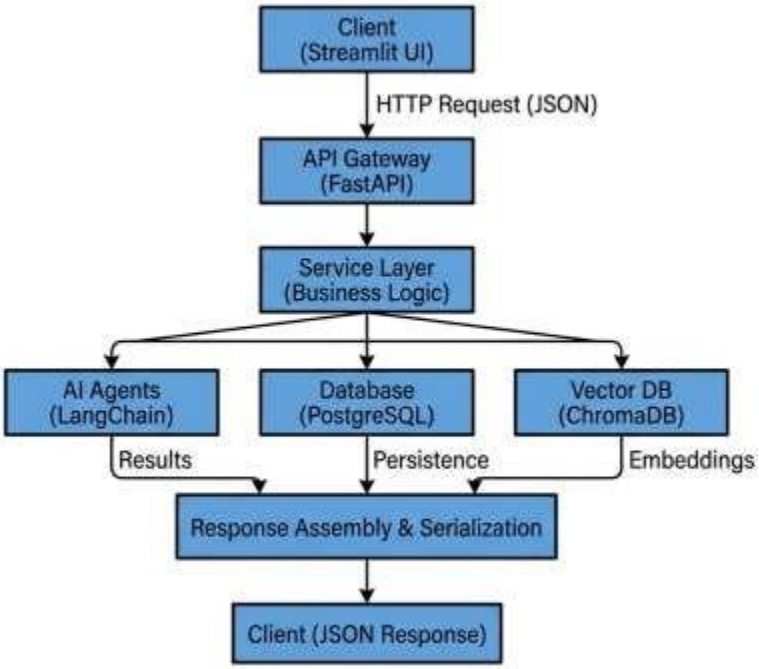
1. **Development Phase**:
    - Developer writes code and commits to Git repository
    - docker-compose.yml defines multi-container application
    - docker-compose up command builds and starts all services
    - Hot-reload enabled for rapid development iteration
2. **Build Phase**:
    - Multi-stage Dockerfile executed
    - Stage 1: Dependencies installed in builder image
    - Stage 2: Runtime image created with minimal footprint
    - Image tagged with version number
3. **CI/CD Pipeline Execution**:
    - Code pushed to GitHub triggers pipeline
    - Jenkins/GitHub Actions pulls code from repository
    - Automated tests executed (unit tests, integration tests)
    - Docker image built and pushed to container registry
    - Kubernetes deployment manifests updated with new image tag
4. **Kubernetes Deployment**:
    - kubectl apply command deploys updated manifests
    - Kubernetes creates new pods with updated container image
    - Rolling update strategy ensures zero-downtime deployment
    - Health checks validate service availability
    - Old pods terminated after successful deployment
5. **Runtime Operation**:
    - Kubernetes manages container lifecycle
    - Auto-scaling triggered based on CPU/memory metrics
    - Load balancer distributes traffic across pod replicas
    - Persistent volumes maintain data across pod restarts
6. **Monitoring and Logging**:
    - Container logs collected by logging agent
    - Metrics exported to monitoring system (Prometheus)
    - Alerts triggered for errors or performance degradation

Health check endpoints monitored continuously

2.3 Information Flow

Data Flow Architecture:

The system implements a unidirectional data flow pattern with clear data transformation stages:



Data Flow Architecture

Information Types and Flow:

Information Type	Source	Destination
Message Content	User Input	Database + Vector DB
Classification Results	LLM API	Database
Entity Extractions	LLM API	Database
Vector Embeddings	Embedding Model	ChromaDB
Draft Responses	LLM API	Database
User Credentials	Login Form	Authentication Service
Analytics Data	Database Aggregation	Dashboard
Container Logs	Docker Containers	Logging Service
Metrics	Application	Monitoring System

Table 1: Information Flow Mapping

2.4 Components Design

Core Application Components:

1. **Streamlit Dashboard** - Real-time message monitoring and manual override interface
2. **FastAPI Backend** - RESTful endpoints for message processing and agent coordination
3. **LangChain Orchestrator** - Manages agent lifecycle, tool routing, and memory persistence
4. **PostgreSQL Database** - Stores message history, user profiles, and audit logs
5. **ChromaDB** - Vector database for semantic search and response template matching

Infrastructure Components:

1. **Docker Containers** - Application isolation with multi-stage build optimization
2. **Kubernetes Cluster** - Auto-scaling pods with Horizontal Pod Autoscaler
3. **Ingress Controller** - SSL termination and routing
4. **Ingress Controller** - SSL termination and traffic routing
5. **Persistent Volumes** - Database and vector store durability

Agent Tooling: External API integrations for email/SMS gateways and notification systems.

2.5 Key Design Considerations

Security Architecture

- **Authentication**: JWT tokens with role-based access control (admin/agent/operator)
- **Data Protection**: Message encryption at rest and in transit (AES-256)
- **API Security**: Rate limiting, input validation, SQL injection prevention
- **Container Security**: Non-root users, read-only filesystems, image vulnerability scanning
- **Secrets Management**: Kubernetes secrets with external vault integration

Scalability Strategy

- **Horizontal Pod Autoscaling**: CPU/memory-based scaling (min: 2, max: 20 replicas)
- **Queue-based Processing**: Celery workers scale independently of API layer
- **Database Sharding**: Tenant isolation through schema separation
- **Cache Layer**: Redis for agent memory and frequent query results

Performance Optimization

- **Multi-stage Docker builds**: 70% image size reduction, 40% faster deployments
- **Connection pooling**: Database and external API connections
- **Async processing**: Non-blocking I/O for AI inference and external calls
- **Lazy loading**: Vector search results cached for 15-minute windows

Reliability Engineering

- **Health checks**: Liveness/readiness probes on all services
- **Circuit breakers**: External API failure isolation
- **Graceful degradation**: Fallback to rule-based triage during AI downtime
- **Data durability**: Multi-AZ PostgreSQL with automated backups

2.6 Infrastructure Catalogue & Data Design

Infrastructure Endpoints

1. **Message Processing API** - Submits messages for AI triage (POST /api/v1/triage)
2. **Dashboard API** - Real-time metrics and message monitoring (GET /api/v1/dashboard)
3. **Agent Status API** - Health and performance monitoring (GET /api/v1/agents)
4. **Knowledge Base API** - Response template management (CRUD operations)
5. **Audit API** - Triage decision history and override tracking

Data Model Design

Primary Entities:

- **Messages**: Raw text, metadata, processing timestamps
- **Classifications**: Urgency scores, intent categories, confidence levels
- **Entities**: Extracted names, amounts, dates, locations
- **Responses**: Generated drafts, human overrides, final outputs
- **AuditTrail**: Complete decision lineage for compliance

Relationships:

- Message (1:N) Classification → Multiple triage results per message
- Message (1:N) Entities → Extracted information per message
- Classification (1:N) Responses → Response chain per classification

Vector Storage: ChromaDB stores message embeddings alongside PostgreSQL metadata for hybrid search capabilities.

3. References

• LangChain Documentation

• Kubernetes Best Practices

• IaC Principles (HashiCorp)

• DevOps Research Assessment

• Project GitHub Repository